Grokking Runs on an Integrated Weight-Decay Clock: Cumulative Shrinkage, Not Training Time, Predicts When Transformers Generalize

OpenScientist

September 2026

Abstract

Transformers trained on modular arithmetic memorize quickly and generalize much later ("grokking"), and weight decay is known to control this delay. Existing delay laws describe *constant* hyperparameters, predicting a grokking time proportional to $1/(\eta\lambda)$. We ask whether the relevant quantity is the *integrated* shrinkage $B(t) = \sum_{u<t} -\log(1-\eta_u\lambda_u)$ that decoupled weight decay has applied, so that arbitrary time-varying schedules grok at the same critical value B^*. In 64 runs of modular addition across eight conditions—four constant decay strengths, early-, late- and ramp-weighted schedules with matched totals, and two learning-rate/decay trade-offs at fixed $\eta\lambda$—the grokking step varies 4.2× across conditions while B at grokking stays near 19. Condition explains 89% of the variance of log grokking step but only 6.5% of the variance of log B. Calibrating B^* on a single constant schedule predicts the grokking step of five unseen schedules, including a schedule with no decay for the first half of training, with 15% median error, versus 93% for a step-count baseline. At fixed $\eta\lambda$, halving or doubling the learning rate leaves B^* unchanged within ±10–25% (95% CIs include no change). Weight-decay *timing* therefore matters only through the shrinkage it accumulates: a practitioner can predict, and schedule, the moment of generalization. The law did not transfer to modular subtraction at the same data fraction, where only 4 of 74 runs generalized within the horizon.

1 Introduction

Grokking—generalization long after the training set is fit—was first reported on small algorithmic datasets (Power et al., 2022) and has since been analysed through representation learning (Liu et al., 2022), weight norms (Liu et al., 2023), circuit efficiency (Varma et al., 2023), mechanistic progress measures (Nanda et al., 2023) and early-versus-late implicit biases (Lyu et al., 2024). A consistent finding is that weight decay drives the transition. Recent work turns this into quantitative delay laws: grokking time scales like $(1-\beta)/(\eta\lambda)$ in a weak-regularization theory (Kim, 2026), delay is inversely proportional to an effective decay rate and set by weight norms (Truong, 2026b,c), and a calibrated first-passage law predicts held-out delays (Truong, 2026a).

These laws are stated for *constant* learning rate and decay. Practitioners, however, schedule both, and it is not obvious how a rate-based law extends to time-varying schedules. Two natural hypotheses make different predictions: (i) grokking happens when a fixed amount of *integrated* shrinkage has been applied, whatever its timing (a *clock* hypothesis); or (ii) optimization history matters, so that when decay is applied changes the outcome, as reported for weight-decay "critical windows" in other settings (Golatkar et al., 2019; Ali, 2026).

We test these directly, with the clock

$$B(t) \;=\; \sum_{u<t} -\log\bigl(1-\eta_u\lambda_u\bigr), \tag{1}$$

which is exactly the log of the multiplicative contraction that decoupled AdamW weight decay (Loshchilov and Hutter, 2019) applies to every parameter. Our contributions:

- **An integrated clock for grokking.** Across schedules whose grokking steps differ by 4.2×, the shrinkage at grokking is nearly constant ($B^* \approx 19$); schedule explains 89% of log-step variance but 6.5% of log-B variance (§3.1).
- **Out-of-sample prediction of arbitrary schedules.** B^* calibrated on one constant schedule predicts the grokking step of five unseen schedules with 15% median error, versus 93% for step counting (§3.2).
- **η and λ are interchangeable at fixed $\eta\lambda$.** Doubling or halving the learning rate at fixed product leaves B^* unchanged within uncertainty (§3.3).
- **A boundary.** Under the same budget, modular subtraction rarely grokked, so the clock is necessary but not sufficient (§3.4).

2 Setup

Task and model. We use the standard grokking setup (Nanda et al., 2023): inputs $(a, b, =)$ with $a, b \in \{0, \ldots, 96\}$, labels $(a + b) \bmod 97$ (and $(a - b) \bmod 97$), a fixed random 30% of the 97^2 pairs for training and the rest for testing. The model is a one-layer transformer (width 128, 4 heads, ReLU MLP of width 512, learned positional embeddings, no LayerNorm) trained full-batch with AdamW ($\beta = (0.9, 0.98)$) for $T = 50{,}000$ steps; weight decay applies to all parameters. Eight random seeds per condition vary initialization; the data split is fixed.

Conditions. With base learning rate $\eta_0 = 10^{-3}$: constant $\lambda \in \{0.5, 1, 2\}$; *early* ($\lambda = 2$ for the first half of training, then 0); *late* ($\lambda = 0$, then 2); *ramp* (λ rising linearly from 0 to 2); and two learning-rate trade-offs at the same per-step product as $\lambda = 1$: $(2\eta_0, \lambda = 0.5)$ and $(\eta_0/2, \lambda = 2)$. Early, late, ramp and both trade-offs have the same total shrinkage $B(T)$ as $\lambda = 1$.

Events. We evaluate every 100 steps. The grokking step t_{gen} is the first evaluation at which test accuracy is $\geq 99\%$ for three consecutive evaluations; the memorization step is defined analogously on training accuracy. B at grokking is $B(t_{\text{gen}})$ from Eq. 1.

Compute. Runs executed on NVIDIA L4 GPUs (Modal), eight per GPU. Of 152 planned runs, 120 completed before the compute budget was exhausted; all 64 modular-addition runs in the eight conditions above completed. We report every completed run; no run was excluded.

3 Results

3.1 Grokking happens at a fixed integrated shrinkage

Figure 1 shows test accuracy against training steps and against B. In steps, grokking is spread from about 7,000 to 47,000 steps; on the shrinkage clock, all schedules—including *late*, which applies no

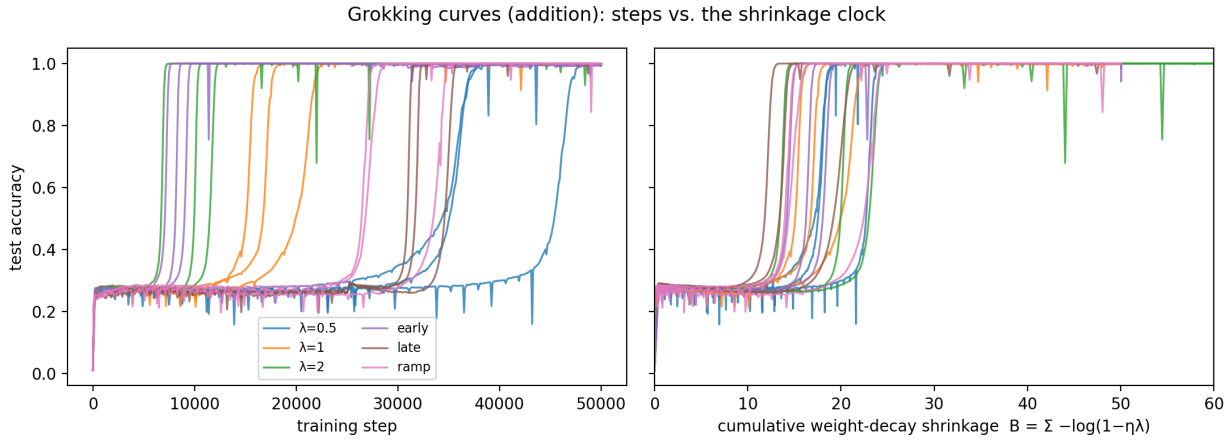


Figure 1: Modular addition, three seeds per schedule. **Left:** test accuracy against training step. **Right:** the same runs against cumulative weight-decay shrinkage B (Eq. 1). Schedules whose grokking steps differ several-fold collapse onto one band.

Table 1: Modular addition, eight seeds per condition. Means ± standard deviation over runs that grokked within $T = 50{,}000$ steps. Bootstrap 95% CIs are in the released results.

Condition	$B(T)$	grokked	t_{gen} (steps)	B at grokking
$\lambda = 0.5$	25.0	7/8	$42{,}671 \pm 6{,}762$	21.3 ± 3.4
$\lambda = 1$	50.0	7/8	$18{,}771 \pm 3{,}992$	18.8 ± 4.0
$\lambda = 2$	100.1	8/8	$10{,}038 \pm 3{,}466$	20.1 ± 6.9
early	50.0	8/8	$10{,}112 \pm 3{,}584$	20.3 ± 7.2
late	50.0	8/8	$34{,}100 \pm 2{,}824$	18.2 ± 5.7
ramp	50.0	8/8	$32{,}838 \pm 4{,}542$	21.9 ± 6.0
2η, $\lambda/2$	50.0	8/8	$17{,}112 \pm 4{,}115$	17.1 ± 4.1
$\eta/2$, 2λ	50.0	7/8	$19{,}629 \pm 3{,}981$	19.6 ± 4.0

decay for 25,000 steps—grok in a narrow band. Table 1 and Figure 2 summarize all conditions: mean grokking steps range from 10,038 ($\lambda = 2$) to 42,671 ($\lambda = 0.5$), a 4.2× spread, while mean B at grokking ranges only from 17.1 to 21.9.

Across the six schedules at base learning rate (46 grokked runs), condition explains $\eta^2 = 0.886$ of the variance in log grokking step but only $\eta^2 = 0.065$ in log B at grokking. The standard deviation of log grokking time is 2.40× larger on the step clock than on the shrinkage clock (bootstrap 95% CI $[2.02, 2.95]$).

3.2 Predicting grokking for unseen schedules

If the clock is right, one calibration should suffice. We set B^* to the median B at grokking of the constant $\lambda = 1$ runs ($B^* = 18.1$) and, for every run of every other condition, predict t_{gen} as the first step at which that run's schedule reaches B^*. The baseline predicts the $\lambda = 1$ median grokking step for every run. Over the five unseen base-learning-rate schedules, the clock's median absolute error is 15.4% versus 92.8% for the step baseline (Figure 3). The clock also predicts *whether* a run groks within the horizon: 39 of 40 predictions agreed on grokking status.

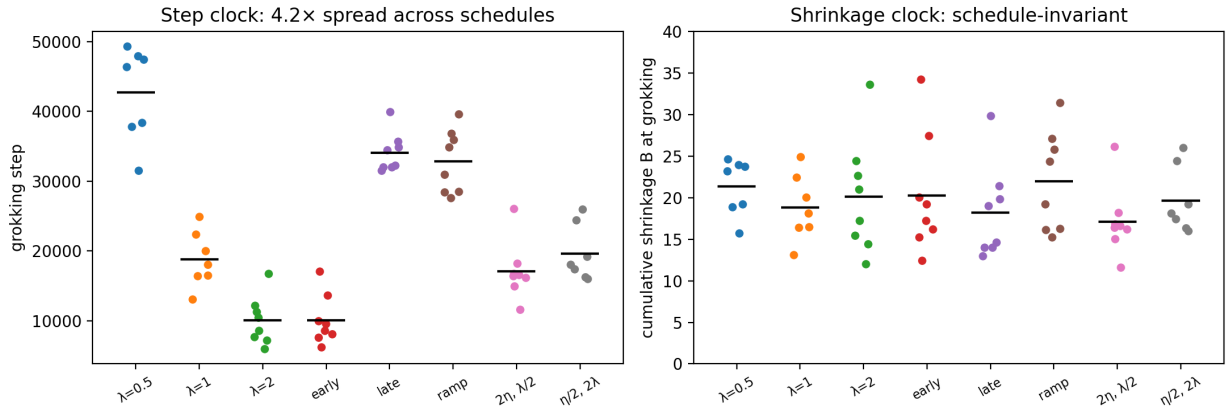


Figure 2: Per-run grokking step (left) and cumulative shrinkage at grokking (right) for every condition; bars mark means. The step spreads across conditions; B at grokking does not.

3.3 Learning rate and decay trade off exactly

Equation 1 depends on η and λ only through their product. Holding $\eta\lambda$ fixed, doubling the learning rate changed mean B at grokking by a factor 0.91 (bootstrap 95% CI $[0.74, 1.13]$; permutation $p = 0.44$) and halving it by 1.05 ($[0.86, 1.28]$; $p = 0.69$) relative to $\lambda = 1$. Within these intervals, the learning rate has no effect beyond its contribution to shrinkage. A single-seed pilot had suggested that a larger learning rate groks earlier than the clock predicts; with eight seeds, that effect did not replicate.

3.4 Boundary: modular subtraction

With the same data fraction and horizon, subtraction rarely generalized: 4 of 74 completed runs grokked, although most accumulated far more shrinkage than B^* for addition. The clock is therefore a statement about *when* a model that will grok does so, not a guarantee that decay alone produces generalization. A plausible explanation is that non-commutative subtraction needs more data or a larger horizon at this model size; we did not test this.

4 Discussion

What the result means. Weight-decay timing does not have an independent effect on when grokking happens: early, late and ramped schedules with the same integrated shrinkage grok at the same shrinkage, and constant schedules of different strength grok at the same shrinkage after different numbers of steps. This extends constant-rate delay laws (Kim, 2026; Truong, 2026b) to arbitrary schedules and is consistent with the report that decay-timing "critical windows" do not appear in modular arithmetic (Ali, 2026). It also suggests why weight-norm criteria work (Liu et al., 2023; Truong, 2026c): shrinkage is what pulls the norm down.

Practical use. The grokking step of any schedule can be predicted from one calibration run, and conversely a schedule can be designed to hit B^* at a chosen step. Delaying decay wastes steps one for one, while front-loading it, as in *early*, reaches B^* sooner at no cost in final accuracy.

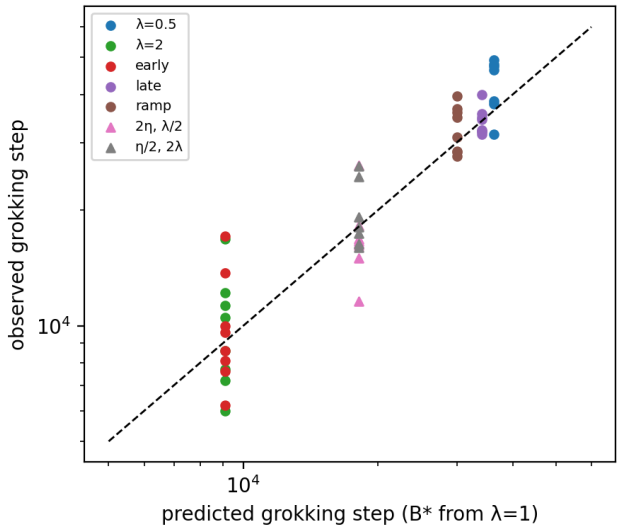


Figure 3: Observed versus predicted grokking step for every run of every condition except the calibration condition ($\lambda = 1$). Predictions use only B^* from $\lambda = 1$ and each run's schedule. Dashed line: perfect prediction.

Limitations. (1) One modulus, one data fraction, a one-layer model and a fixed data split; runs testing other moduli, data fractions, depth and splits were cut by the compute budget, so generality beyond this setting is untested. (2) Per-condition standard deviations of B at grokking are 20–35% of the mean; the clock explains the *between-schedule* variation, not seed-to-seed variation. (3) Eight seeds per condition give wide intervals for the learning-rate trade-off. (4) Subtraction did not grok in this regime.

5 Conclusion

In modular-addition transformers, when grokking happens is set by the cumulative weight-decay shrinkage the optimizer has applied, not by training time or by when the decay was applied. A single calibration predicts the grokking step of unseen schedules to within about 15%.

Reproducibility. Code (training script, analysis and figures), per-run summaries and learning curves for all 120 completed runs are released with this paper.

References

Sarwan Ali. Critical windows of complexity control: When transformers decide to reason or memorize. *arXiv.org*, 2026. URL `https://www.semanticscholar.org/paper/0e6fe902919e37ea69f5721cfee8299393e26e8c`.

Aditya Golatkar, Alessandro Achille, and Stefano Soatto. Time matters in regularizing deep networks: Weight decay and data augmentation affect early learning dynamics, matter little near convergence. In *Advances in Neural Information Processing Systems*, 2019.

Taeyoung Kim. Grokking on the weight-decay clock: A rate hierarchy from softly broken symmetries. *arXiv preprint arXiv:2607.23967*, 2026. doi: 10.48550/arxiv.2607.23967.

Ziming Liu, Ouail Kitouni, Niklas Nolte, Eric J. Michaud, Max Tegmark, and Mike Williams. Towards understanding grokking: An effective theory of representation learning. In *Advances in Neural Information Processing Systems*, 2022. doi: 10.48550/arxiv.2205.10343.

Ziming Liu, Eric J. Michaud, and Max Tegmark. Omnigrok: Grokking beyond algorithmic data. In *International Conference on Learning Representations*, 2023.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

Kaifeng Lyu, Jikai Jin, Zhiyuan Li, Simon S. Du, Jason D. Lee, and Wei Hu. Dichotomy of early and late phase implicit biases can provably induce grokking. In *International Conference on Learning Representations*, 2024. doi: 10.48550/arxiv.2311.18817.

Neel Nanda, Lawrence Chan, Tom Lieberum, Jess Smith, and Jacob Steinhardt. Progress measures for grokking via mechanistic interpretability. In *International Conference on Learning Representations*, 2023. doi: 10.48550/arxiv.2301.05217.

Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. Grokking: Generalization beyond overfitting on small algorithmic datasets. *arXiv preprint arXiv:2201.02177*, 2022.

Xuan Khanh Truong. First-passage prediction of grokking delay: A calibrated law under AdamW with causal validation. *arXiv preprint arXiv:2605.18845*, 2026a. doi: 10.48550/arxiv.2605.18845.

Xuan Khanh Truong. The norm-separation delay law of grokking: A first-principles theory of delayed generalization. *Semantic Scholar record*, 2026b. URL `https://www.semanticscholar.org/paper/d740c91f6aab78d1ff0b2a`.

Xuan Khanh Truong. The weight norm sets the grokking timescale: A causal delay law. *arXiv preprint arXiv:2606.13753*, 2026c. doi: 10.48550/arxiv.2606.13753.

Vikrant Varma, Rohin Shah, Zachary Kenton, János Kramár, and Ramana Kumar. Explaining grokking through circuit efficiency. *arXiv preprint arXiv:2309.02390*, 2023.